

Problem pięciu filozofów

Szymon Iwaniuk

Teoria Współbieżności 2025/2026, laboratorium 5
8 listopada 2026

Spis treści

1 Wstęp **3**
1.1 Treść zadania . . . 3
1.2 Opis rozwiązania . . . 3

2 Specyfikacja narzędzi i sprzętu **4**
2.1 Metoda pomiaru . . . 4

3 Algorytmy **5**
3.1 Rozwiązanie trywialne . . . 6
3.2 Rozwiązanie z widelcami podnoszonymi jednocześnie . . . 6
3.3 Rozwiązanie z lokajem . . . 7
3.4 Rozwiązanie asymetryczne (autorskie) . . . 7

4 Implementacja **8**

5 Wyniki **17**
5.1 Blokada w rozwiązaniu trywialnym . . . 17
5.2 Przepustowość . . . 17
5.3 Czas oczekiwania . . . 18
5.4 Sprawiedliwość . . . 19
5.5 Zagłodzenie przez sąsiadów . . . 20

6 Wnioski **20**

1 Wstęp

1.1 Treść zadania

Celem ćwiczenia jest zapoznanie się z jednym z klasycznych problemów teorii współbieżności – problemem pięciu filozofów (autor E.W. Dijkstra).



Rysunek 1: Problem pięciu filozofów

- Każdy filozof zajmuje się głównie myśleniem.
- Od czasu do czasu potrzebuje zjeść.
- Do jedzenia potrzebne mu są oba widelce po jego prawej i lewej stronie.
- Jedzenie trwa skończoną (ale nieokreśloną z góry) ilość czasu, po czym filozof widelce odkłada i wraca do myślenia.
- Cykl powtarza się od początku.

Ćwiczenia (na podstawie szkieletu programu `Fil5mon.java`):

a) Zaimplementować trywialne rozwiązanie z symetrycznymi filozofami. Zaobserwować problem blokady.

b) Zaimplementować rozwiązanie z widelcami podnoszonymi jednocześnie. Jaki problem może tutaj wystąpić?

c) Zaimplementować rozwiązanie z lokajem.

d) Wykonać pomiary dla każdego rozwiązania i wywnioskować, co ma wpływ na wydajność każdego rozwiązania.

Dodatkowe zadanie: zaproponować autorskie rozwiązanie, inne niż powyższe.

1.2 Opis rozwiązania

Porównujemy cztery implementacje stołu, wszystkie oparte na monitorach Javy (`synchronized`, `wait()`, `notify()`):

- **a) trywialne** – każdy filozof podnosi najpierw lewy, potem prawy widelec,
- **b) oba widelce naraz** – jeden monitor stołu, filozof czeka, aż oba jego widelce będą wolne, i zabiera je w jednym kroku,
- **c) lokaj** – jak a), ale do stołu jednocześnie dopuszczonych jest co najwyżej $n - 1$ filozofów,
- **asymetryczne (autorskie)** – każdy filozof podnosi najpierw widelec o mniejszym numerze, więc ostatni filozof podnosi widelce w odwrotnej kolejności niż pozostali.

Program działa dla dowolnej liczby filozofów n, a nie tylko dla pięciu, dzięki czemu można zbadać, jak rozwiązania skalują się z wielkością stołu.

Razem ze sprawozdaniem zawarty jest kod w języku `Java` zawierający wszystkie implementacje, program wykonujący pomiary oraz skrypt służący do generowania zawartych w dokumencie wykresów.

2 Specyfikacja narzędzi i sprzętu

Program został napisany w języku `Java 27` i uruchomiony na maszynie wirtualnej `OpenJDK`, na komputerze z procesorem `Apple M4` (architektura `ARM64`, 10 rdzeni: 4 wydajnościowe i 6 energooszczędnych), `16 GB` zunifikowanej pamięci `LPDDR5` i systemem operacyjnym `macOS 26`. Do wygenerowania wykresów użyty został `Python` z biblioteką `matplotlib`.

2.1 Metoda pomiaru

Filozof w pętli myśli, staje się głodny, zdobywa oba widelce, je i odkłada widelce. Czasy myślenia i jedzenia są losowane z rozkładu jednostajnego na przedziale $[0, 2t)$, gdzie t to średni czas danego scenariusza. Opóźnienie realizujemy przez `LockSupport.parkNanos`, więc wątek nie zajmuje procesora. Badamy cztery scenariusze (średni czas myślenia / jedzenia):

- **0/0** – bez myślenia i jedzenia, mierzymy wyłącznie koszt synchronizacji,
- **0/1** – myślenie 0 ms, jedzenie 1 ms, filozofowie są cały czas głodni (największa rywalizacja),
- **1/1** – myślenie 1 ms, jedzenie 1 ms,
- **10/1** – myślenie 10 ms, jedzenie 1 ms (mała rywalizacja).

Każdy przebieg trwa 1 s, a wszystkie wątki startują jednocześnie po zwolnieniu zatrzasku `CountDownLatch`. Każdy filozof liczy swoje posiłki oraz czas oczekiwania na widelce, czyli czas od zgłoszenia głodu do rozpoczęcia jedzenia (`System.nanoTime()`). Na tej podstawie wyznaczamy przepustowość (liczba posiłków na sekundę), średni i maksymalny czas oczekiwania oraz indeks Jaina liczby posiłków

$$J(x_1, \ldots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2},$$

gdzie x_i to liczba posiłków i-tego filozofa. $J = 1$, gdy wszyscy zjedli tyle samo razy, a $J = \frac{1}{n}$, gdy jadł tylko jeden filozof. Przed pomiarami wykonujemy kilka przebiegów rozgrzewających kompilator JIT, a każdą konfigurację uruchamiamy trzy razy i bierzemy medianę. Parametry: $n \in \{5, 10, 20, 50, 100\}$.

Rozwiązanie trywialne badamy osobno: przebieg kończy się w chwili wykrycia blokady (albo po 1 s), a każdą konfigurację $n \in \{5, 10, 20, 50\}$ uruchamiamy 20 razy. Blokadę wykrywamy dokładnie: stół liczy filozofów, którzy trzymają lewy widelec i czekają na prawy. Gdy jest ich n, wszystkie widelce są zajęte i żaden filozof nie może kontynuować.

Ostatni eksperyment sprawdza zagłodzenie: przy $n = 5$ filozofowie F1 i F3 („żarłoki") w ogóle nie myślą i jedzą średnio 2 ms, a pozostali myślą 10 ms i jedzą 1 ms. Przebieg trwa 3 s.

W każdym przebiegu filozof po zdobyciu widelców sprawdza, czy żaden z jego sąsiadów nie je. Takie zdarzenie przerwałoby pomiary wyjątkiem – we wszystkich przebiegach nie wystąpiło.

3 Algorytmy

Filozof i korzysta z widelca i (lewego) i widelca $(i + 1) \bmod n$ (prawego). Wszystkie rozwiązania różnią się tylko procedurami ACQUIRE i RELEASE, pętla filozofa jest wspólna.

Pętla filozofa

```
1: Procedura FILOZOF(i)
2:     Powtarzaj
3:         myśl
4:         t_0 ← NOW()
5:         ACQUIRE(i)
6:         zapisz czas oczekiwania NOW() − t_0
7:         jedz
8:         RELEASE(i)
```

Widelec w rozwiązaniach a), c) i asymetrycznym jest monitorem działającym jak semafor binarny.

Widelec

```
1: Procedura TAKE(w) ▷ synchronized
2:     Dopóki w.zajęty powtarzaj
3:         WAIT()
4:     w.zajęty ← true
5: Procedura PUT(w) ▷ synchronized
6:     w.zajęty ← false,  NOTIFY()
```

3.1 Rozwiązanie trywialne

Rozwiązanie trywialne

```
1: Procedura ACQUIRE(i)
2:     TAKE(widelec[i])
3:     TAKE(widelec[(i + 1) mod n])
4: Procedura RELEASE(i)
5:     PUT(widelec[(i + 1) mod n]),  PUT(widelec[i])
```

Jeżeli wszyscy filozofowie jednocześnie wykonają wiersz 2, każdy trzyma swój lewy widelec, a jego prawy widelec jest lewym widelcem sąsiada. Spełnione są wszystkie cztery warunki Coffmana: wzajemne wykluczanie, trzymanie zasobu podczas oczekiwania, brak wywłaszczania oraz cykl oczekiwania $F_0 \to F_1 \to \cdots \to F_{n-1} \to F_0$. Dochodzi do blokady (zakleszczenia).

3.2 Rozwiązanie z widelcami podnoszonymi jednocześnie

Widelce podnoszone jednocześnie

```
1: Procedura ACQUIRE(i) ▷ synchronized na całym stole
2:     Dopóki zajęty[i] ∨ zajęty[(i + 1) mod n] powtarzaj
3:         WAIT()
4:     zajęty[i] ← true,  zajęty[(i + 1) mod n] ← true
5: Procedura RELEASE(i) ▷ synchronized na całym stole
6:     zajęty[i] ← false,  zajęty[(i + 1) mod n] ← false
7:     NOTIFYALL() ▷ obaj sąsiedzi mogą czekać
```

Filozof nigdy nie trzyma jednego widelca, czekając na drugi, więc warunek „trzymaj i czekaj” nie jest spełniony i blokada jest niemożliwa. Pojawia się za to **zagłodzenie**: filozof i może jeść tylko wtedy, gdy jednocześnie nie je żaden z jego sąsiadów. Jeżeli sąsiedzi jedzą na zmianę tak, że zawsze je co najmniej jeden z nich, filozof i czeka w nieskończoność, mimo że cały system działa. Dodatkowo cały stół jest jednym monitorem, a NOTIFYALL budzi wszystkich czekających, chociaż jeść mogą co najwyżej dwaj sąsiedzi zwalniającego.

3.3 Rozwiązanie z lokajem

Rozwiązanie z lokajem

```
1: Procedura ACQUIRE(i)
2:     synchronized(lokaj): dopóki przy stole = n − 1 WAIT();  przy stole ← przy stole + 1
3:     TAKE(widelec[i])
4:     TAKE(widelec[(i + 1) mod n])
5: Procedura RELEASE(i)
6:     PUT(widelec[(i + 1) mod n]),  PUT(widelec[i])
7:     synchronized(lokaj): przy stole ← przy stole − 1,  NOTIFY()
```

Przy stole jest co najwyżej $n-1$ filozofów, a widelców jest n. Z zasady szufladkowej co najmniej jeden filozof przy stole ma dostęp do obu swoich widelców, więc cykl oczekiwania nie może się zamknąć. Lokaj jest jednak wspólnym monitorem, przez który przechodzi każdy posiłek.

3.4 Rozwiązanie asymetryczne (autorskie)

Rozwiązanie asymetryczne (hierarchia zasobów)

```
1: Procedura ACQUIRE(i)
2:     a ← min(i, (i + 1) mod n),  b ← max(i, (i + 1) mod n)
3:     TAKE(widelec[a]),  TAKE(widelec[b])
4: Procedura RELEASE(i)
5:     PUT(widelec[b]),  PUT(widelec[a])
```

Widelce są ponumerowane, a każdy filozof podnosi je w kolejności rosnących numerów. Dla filozofów $0, \ldots, n-2$ oznacza to lewy, potem prawy widelec, a filozof $n-1$ (widelce $n-1$ i 0) podnosi najpierw prawy. W cyklu oczekiwania każdy wątek czekałby na widelec o większym numerze niż ten, który trzyma, co jest niemożliwe w zamkniętym cyklu. Rozwiązanie nie wymaga żadnego wspólnego monitora – każdy widelec synchronizuje tylko dwóch sąsiadów.

4 Implementacja

Poniżej znajduje się pełny kod programu. Każda klasa jest w osobnym pliku w katalogu `src`.
Klasa **Fork** to widelec ze szkieletu programu (`Widelec`) zrealizowany jako monitor.

```
/** Fork as a Java monitor, like Widelec from the skeleton: take() waits until the fork is free. */
class Fork {
    private boolean taken = false;

    synchronized void take() throws InterruptedException {
        while (taken) wait();
        taken = true;
    }

    synchronized void put() {
        taken = false;
        notify();
    }
}
```

Kod 1: Klasa **Fork** (`src/Fork.java`)

Klasa **Table** jest wspólną bazą wszystkich rozwiązań. Metoda `create` wybiera implementację po nazwie.

```
/**
 * Common base of all solutions. Philosopher i uses fork i (left) and fork (i+1) mod n (right);
 * acquire(i) returns when philosopher i holds both forks, release(i) puts them back.
 */
abstract class Table {
    final int n;

    Table(int n) {
        this.n = n;
    }

    int left(int i) {
        return i;
    }

    int right(int i) {
        return (i + 1) % n;
    }

    abstract void acquire(int i) throws InterruptedException;

    abstract void release(int i);

    /** True when the table is certainly in a deadlock (only the naive solution can be). */
    boolean deadlocked() {
        return false;
    }

    static Fork[] forks(int n) {
        Fork[] forks = new Fork[n];
```

```
31          for (int i = 0; i < n; i++) forks[i] = new Fork();
32          return forks;
33      }
34
35      static Table create(String impl, int n) {
36          return switch (impl) {
37              case "naive" -> new NaiveTable(n);
38              case "both" -> new BothTable(n);
39              case "waiter" -> new WaiterTable(n);
40              case "asym" -> new AsymmetricTable(n);
41              default -> throw new IllegalArgumentException(impl);
42          };
43      }
44  }
```

Kod 2: Klasa **Table** (`src/Table.java`)

Klasa **NaiveTable** to rozwiązanie trywialne a). Licznik `holdingOne` służy wyłącznie do wykrywania blokady.

```
1   import java.util.concurrent.atomic.AtomicInteger;
2
3   /** a) Symmetric philosophers: left fork first, then right fork. Can deadlock. */
4   class NaiveTable extends Table {
5       private final Fork[] forks;
6       // philosophers holding their left fork and not yet the right one
7       private final AtomicInteger holdingOne = new AtomicInteger();
8
9       NaiveTable(int n) {
10          super(n);
11          forks = forks(n);
12      }
13
14      @Override
15      void acquire(int i) throws InterruptedException {
16          forks[left(i)].take();
17          holdingOne.incrementAndGet();
18          forks[right(i)].take();
19          holdingOne.decrementAndGet();
20      }
21
22      @Override
23      void release(int i) {
24          forks[right(i)].put();
25          forks[left(i)].put();
26      }
27
28      /** All n philosophers hold one fork each and wait for the other: nobody can ever continue. */
29      @Override
30      boolean deadlocked() {
31          return holdingOne.get() == n;
32      }
33  }
```

Kod 3: Klasa **NaiveTable** (`src/NaiveTable.java`)

Klasa **BothTable** to rozwiązanie b) z widelcami podnoszonymi jednocześnie.

```
/**
 * b) Both forks taken at once: one monitor for the whole table, a philosopher waits
 * until both forks are free and takes them in one step. No deadlock, but
 * a philosopher whose neighbours eat in turns can starve.
 */
class BothTable extends Table {
    private final boolean[] taken;

    BothTable(int n) {
        super(n);
        taken = new boolean[n];
    }

    @Override
    synchronized void acquire(int i) throws InterruptedException {
        while (taken[left(i)] || taken[right(i)]) wait();
        taken[left(i)] = true;
        taken[right(i)] = true;
    }

    @Override
    synchronized void release(int i) {
        taken[left(i)] = false;
        taken[right(i)] = false;
        notifyAll(); // both neighbours may be waiting
    }
}
```

Kod 4: Klasa **BothTable** (`src/BothTable.java`)

Klasy **Waiter** i **WaiterTable** to rozwiązanie c) z lokajem.

```
/** Waiter (lokaj) lets at most limit philosophers try to take forks at the same time. */
class Waiter {
    private final int limit;
    private int seated = 0;

    Waiter(int limit) {
        this.limit = limit;
    }

    synchronized void enter() throws InterruptedException {
        while (seated == limit) wait();
        seated++;
    }

    synchronized void leave() {
        seated--;
        notify(); // exactly one place became free
```

```
    }
}
```

Kod 5: Klasa **Waiter** (`src/Waiter.java`)

```
/** c) Naive fork order guarded by a waiter that admits at most n - 1 philosophers. */
class WaiterTable extends Table {
    private final Fork[] forks;
    private final Waiter waiter;

    WaiterTable(int n) {
        super(n);
        forks = forks(n);
        waiter = new Waiter(n - 1);
    }

    @Override
    void acquire(int i) throws InterruptedException {
        waiter.enter();
        forks[left(i)].take();
        forks[right(i)].take();
    }

    @Override
    void release(int i) {
        forks[right(i)].put();
        forks[left(i)].put();
        waiter.leave();
    }
}
```

Kod 6: Klasa **WaiterTable** (`src/WaiterTable.java`)

Klasa **AsymmetricTable** to rozwiązanie autorskie z hierarchią widelców.

```
/**
 * Own solution: resource ordering. Every philosopher takes the fork with the lower number
 * first, so the last one (forks n-1 and 0) takes the right fork first. No central monitor.
 */
class AsymmetricTable extends Table {
    private final Fork[] forks;

    AsymmetricTable(int n) {
        super(n);
        forks = forks(n);
    }

    @Override
    void acquire(int i) throws InterruptedException {
        forks[Math.min(left(i), right(i))].take();
        forks[Math.max(left(i), right(i))].take();
    }

    @Override
```

```
20      void release(int i) {
21          forks[Math.max(left(i), right(i))].put();
22          forks[Math.min(left(i), right(i))].put();
23      }
24  }
```

Kod 7: Klasa **AsymmetricTable** (`src/AsymmetricTable.java`)

Klasa **Philosopher** to wątek filozofa ze szkieletu (`Filozof`) rozszerzony o pomiary i sprawdzenie, czy sąsiedzi nie jedzą jednocześnie.

```
1   import java.util.SplittableRandom;
2   import java.util.concurrent.CountDownLatch;
3   import java.util.concurrent.atomic.AtomicIntegerArray;
4   import java.util.concurrent.locks.LockSupport;
5
6   /**
7    * Thinks and eats for random times (uniform on [0, 2 * mean)) until stop is set,
8    * recording the number of meals and the time spent waiting for forks.
9    */
10  class Philosopher extends Thread {
11      private final int id;
12      private final Table table;
13      private final long thinkNs, eatNs;
14      private final SplittableRandom rnd;
15      private final AtomicIntegerArray eating; // shared, used to check mutual exclusion
16      private final CountDownLatch start;
17      private volatile boolean stop = false;
18      long meals = 0, waitSumNs = 0, waitMaxNs = 0;
19
20      Philosopher(int id, Table table, long thinkNs, long eatNs, long seed,
21                  AtomicIntegerArray eating, CountDownLatch start) {
22          this.id = id;
23          this.table = table;
24          this.thinkNs = thinkNs;
25          this.eatNs = eatNs;
26          this.rnd = new SplittableRandom(seed);
27          this.eating = eating;
28          this.start = start;
29      }
30
31      void finish() {
32          stop = true;
33      }
34
35      private void pause(long meanNs) {
36          if (meanNs == 0) return;
37          long end = System.nanoTime() + rnd.nextLong(2 * meanNs);
38          long left;
39          while ((left = end - System.nanoTime()) > 0) LockSupport.parkNanos(left);
40      }
41
42      @Override
43      public void run() {
```

```
44          int n = table.n;
45          try {
46              start.await();
47              while (!stop) {
48                  pause(thinkNs);                                    // thinking
49                  long t0 = System.nanoTime();
50                  table.acquire(id);                                 // hungry
51                  long wait = System.nanoTime() - t0;
52                  eating.set(id, 1);
53                  if (eating.get((id + 1) % n) == 1 || eating.get((id + n - 1) % n) == 1)
54                      throw new IllegalStateException("neighbours of " + id + " eat at the same time");
55                  meals++;
56                  waitSumNs += wait;
57                  waitMaxNs = Math.max(waitMaxNs, wait);
58                  pause(eatNs);                                      // eating
59                  eating.set(id, 0);
60                  table.release(id);
61              }
62          } catch (InterruptedException e) {
63              // stopped while waiting, only happens after a deadlock
64          }
65      }
66  }
```

Kod 8: Klasa **Philosopher** (`src/Philosopher.java`)

Klasa **Main** wykonuje wszystkie trzy eksperymenty i zapisuje wyniki do plików CSV, z których skrypt w Pythonie rysuje wykresy.

```
1   import java.io.FileWriter;
2   import java.io.IOException;
3   import java.io.PrintWriter;
4   import java.util.Arrays;
5   import java.util.concurrent.CountDownLatch;
6   import java.util.concurrent.atomic.AtomicIntegerArray;
7   import java.util.concurrent.atomic.AtomicReference;
8   import java.util.concurrent.locks.LockSupport;
9
10  /**
11   * Three experiments, results in csv/:
12   * runs.csv       - deadlock-free solutions, throughput, waiting time and fairness vs n and scenario,
13   * deadlock.csv   - naive solution, time until deadlock,
14   * starvation.csv - n = 5 with two greedy philosophers, meals of every philosopher.
15   */
16  public class Main {
17      static final String[] IMPLS = {"both", "waiter", "asym"};
18      static final int[] SIZES = {5, 10, 20, 50, 100};
19      static final int[] DEADLOCK_SIZES = {5, 10, 20, 50};
20      static final long MS = 1_000_000L;
21      /** name, mean thinking time, mean eating time [ns] */
22      record Scenario(String name, long thinkNs, long eatNs) {}
23      static final Scenario[] SCENARIOS = {
24              new Scenario("0/0", 0, 0),
25              new Scenario("0/1", 0, MS),
```

```
26              new Scenario("1/1", MS, MS),
27              new Scenario("10/1", 10 * MS, MS),
28      };
29      static final long RUN_NS = 1000 * MS;
30      static final int REPS = 3;
31      static final int DEADLOCK_REPS = 20;
32      static final long STARVE_NS = 3000 * MS;
33
34      record Result(long timeNs, boolean deadlock, long[] meals, double[] perMeanUs, double[] perMaxUs) {
35          long totalMeals() {
36              return Arrays.stream(meals).sum();
37          }
38
39          double meanWaitUs() {
40              double sum = 0;
41              for (int i = 0; i < meals.length; i++) sum += perMeanUs[i] * meals[i];
42              return totalMeals() == 0 ? 0 : sum / totalMeals();
43          }
44
45          double maxWaitUs() {
46              return Arrays.stream(perMaxUs).max().orElse(0);
47          }
48
49          /** Jain's index of meals: 1 when everybody ate the same number of times, 1/n when one ate. */
50          double jain() {
51              double sum = 0, sumSq = 0;
52              for (long m : meals) {
53                  sum += m;
54                  sumSq += (double) m * m;
55              }
56              return sumSq == 0 ? 0 : sum * sum / (meals.length * sumSq);
57          }
58      }
59
60      static Result run(String impl, long[] think, long[] eat, long limitNs, long seed) throws
        ↪ InterruptedException {
61          int n = think.length;
62          Table table = Table.create(impl, n);
63          AtomicIntegerArray eating = new AtomicIntegerArray(n);
64          AtomicReference<Throwable> error = new AtomicReference<>();
65          CountDownLatch start = new CountDownLatch(1);
66          Philosopher[] ph = new Philosopher[n];
67          for (int i = 0; i < n; i++) {
68              ph[i] = new Philosopher(i, table, think[i], eat[i], seed * 1_000 + i, eating, start);
69              ph[i].setUncaughtExceptionHandler((t, e) -> error.compareAndSet(null, e));
70              ph[i].start();
71          }
72          long t0 = System.nanoTime();
73          start.countDown();
74          boolean deadlock = false;
75          while (System.nanoTime() - t0 < limitNs) {
76              if (table.deadlocked()) {
77                  deadlock = true;
```

```
78                  break;
79              }
80              LockSupport.parkNanos(100_000);
81          }
82          long elapsed = System.nanoTime() - t0;
83          for (Philosopher p : ph) p.finish();
84          if (deadlock) for (Philosopher p : ph) p.interrupt();
85          for (Philosopher p : ph) p.join();
86          if (error.get() != null) throw new IllegalStateException(impl, error.get());
87
88          long[] meals = new long[n];
89          double[] mean = new double[n], max = new double[n];
90          for (int i = 0; i < n; i++) {
91              meals[i] = ph[i].meals;
92              mean[i] = meals[i] == 0 ? 0 : ph[i].waitSumNs / 1e3 / meals[i];
93              max[i] = ph[i].waitMaxNs / 1e3;
94          }
95          return new Result(elapsed, deadlock, meals, mean, max);
96      }
97
98      static long[] same(int n, long v) {
99          long[] a = new long[n];
100         Arrays.fill(a, v);
101         return a;
102     }
103
104     public static void main(String[] args) throws IOException, InterruptedException {
105         String out = args.length > 0 ? args[0] : "csv";
106
107         // JIT warm-up, results discarded
108         for (String impl : IMPLS)
109             for (int i = 0; i < 3; i++) run(impl, same(10, 0), same(10, 0), 300 * MS, i);
110
111         try (PrintWriter w = new PrintWriter(new FileWriter(out + "/runs.csv"))) {
112             w.println("impl,scenario,n,rep,time_ms,meals,throughput,mean_wait_us,max_wait_us,jain");
113             for (Scenario s : SCENARIOS)
114                 for (int n : SIZES)
115                     for (String impl : IMPLS)
116                         for (int r = 0; r < REPS; r++) {
117                             Result res = run(impl, same(n, s.thinkNs()), same(n, s.eatNs()), RUN_NS, r);
118                             if (res.deadlock()) throw new IllegalStateException(impl + " deadlocked");
119                             double sec = res.timeNs() / 1e9;
120                             w.printf("%s,%s,%d,%d,%.3f,%d,%.1f,%.3f,%.3f,%.4f%n", impl, s.name(), n, r,
121                                     res.timeNs() / 1e6, res.totalMeals(), res.totalMeals() / sec,
122                                     res.meanWaitUs(), res.maxWaitUs(), res.jain());
123                             System.out.printf("%-5s n=%-3d %-6s rep %d: %10.0f meals/s, wait %9.1f us, J
    ↪ =%.3f%n",
124                                     s.name(), n, impl, r, res.totalMeals() / sec, res.meanWaitUs(), res.
    ↪ jain());
125                         }
126         }
127
128         try (PrintWriter w = new PrintWriter(new FileWriter(out + "/deadlock.csv"))) {
```

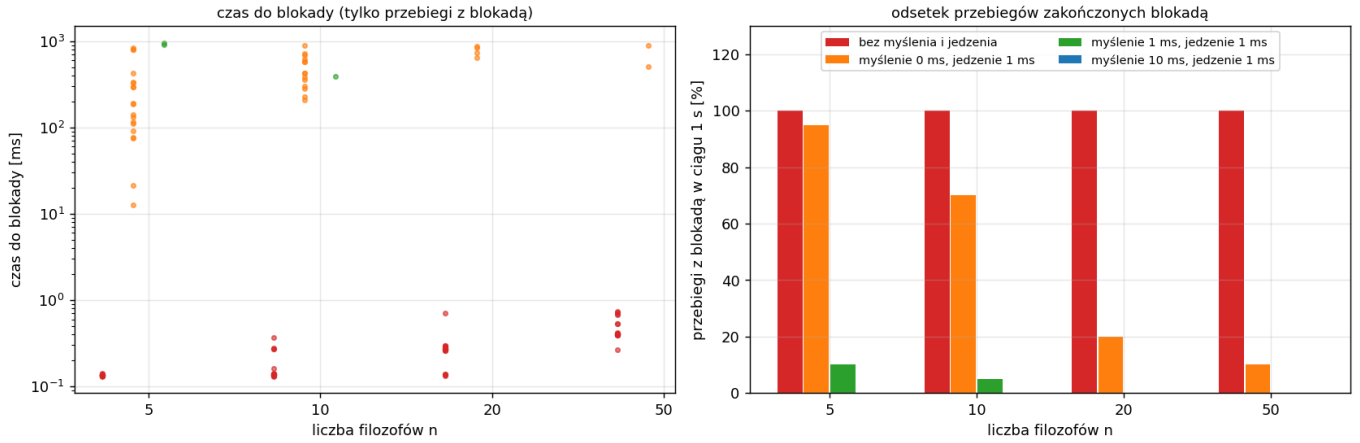
```
129             w.println("scenario,n,rep,deadlock,time_ms,meals");
130             for (Scenario s : SCENARIOS)
131                 for (int n : DEADLOCK_SIZES)
132                     for (int r = 0; r < DEADLOCK_REPS; r++) {
133                         Result res = run("naive", same(n, s.thinkNs()), same(n, s.eatNs()), RUN_NS, r);
134                         w.printf("%s,%d,%d,%d,%.3f,%d%n", s.name(), n, r, res.deadlock() ? 1 : 0,
135                                 res.timeNs() / 1e6, res.totalMeals());
136                         System.out.printf("%-5s n=%-3d naive  rep %2d: %s after %8.2f ms, %d meals%n", s
    ↪ .name(), n, r,
137                                 res.deadlock() ? "DEADLOCK" : "no deadlock", res.timeNs() / 1e6, res.
    ↪ totalMeals());
138                     }
139         }
140
141         // philosophers 1 and 3 never think and eat 2 ms, the others think 10 ms and eat 1 ms
142         long[] think = {10 * MS, 0, 10 * MS, 0, 10 * MS};
143         long[] eat = {MS, 2 * MS, MS, 2 * MS, MS};
144         try (PrintWriter w = new PrintWriter(new FileWriter(out + "/starvation.csv"))) {
145             w.println("impl,rep,philosopher,meals,mean_wait_us,max_wait_us");
146             for (String impl : IMPLS)
147                 for (int r = 0; r < REPS; r++) {
148                     Result res = run(impl, think, eat, STARVE_NS, r);
149                     for (int i = 0; i < 5; i++)
150                         w.printf("%s,%d,%d,%d,%.3f,%.3f%n", impl, r, i, res.meals()[i], res.perMeanUs()[
    ↪ i], res.perMaxUs()[i]);
151                     System.out.printf("starvation %-6s rep %d: meals %s%n", impl, r, Arrays.toString(res
    ↪ .meals()));
152                 }
153         }
154     }
155 }
```

Kod 9: Klasa **Main** (`src/Main.java`)

5 Wyniki

5.1 Blokada w rozwiązaniu trywialnym

Rysunek 2 pokazuje czas do wystąpienia blokady oraz odsetek przebiegów, w których blokada wystąpiła w ciągu 1 s.



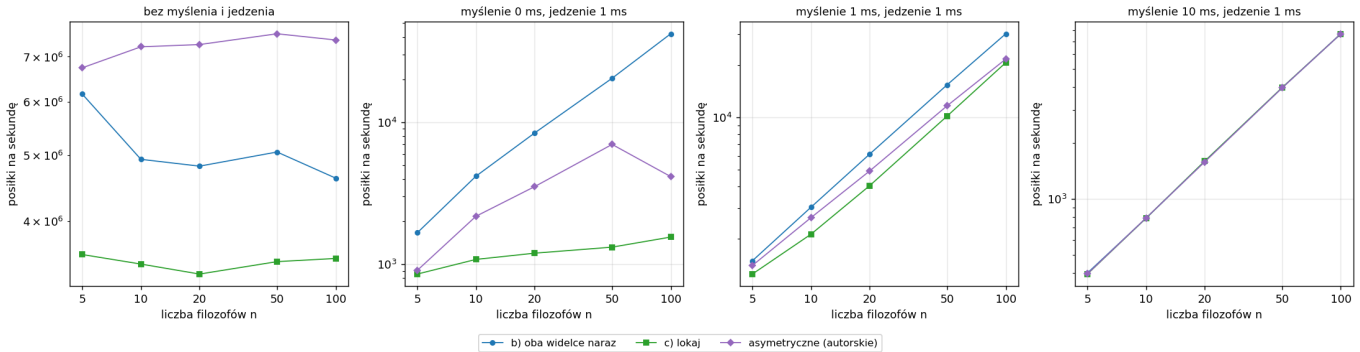
Rysunek 2: Blokada w rozwiązaniu trywialnym (20 przebiegów na konfigurację)

Bez myślenia i jedzenia blokada wystąpiła w każdym przebiegu, a mediana czasu do blokady wynosi 0,14 ms dla $n = 5$ i 0,41 ms dla $n = 50$ (filozofowie zdążyli zjeść łącznie odpowiednio 98 i 1577 razy). Przy ciągle głodnych filozofach i jedzeniu 1 ms blokada wystąpiła w 95% przebiegów dla $n = 5$ (mediana 186 ms), ale tylko w 20% dla $n = 20$ i 10% dla $n = 50$. Gdy filozofowie myślą 1 ms, blokada w ciągu sekundy zdarzała się rzadko (10% dla $n = 5$), a przy 10 ms myślenia nie wystąpiła ani razu.

Blokada wymaga, żeby wszyscy filozofowie jednocześnie trzymali lewy widelec. Im krótsze myślenie i im mniej filozofów, tym większe szanse na takie ułożenie, dlatego rozwiązanie trywialne wydaje się działać poprawnie przy małej rywalizacji. Nie jest jednak poprawne – blokada jest tylko kwestią czasu.

5.2 Przepustowość

Rysunek 3 pokazuje liczbę posiłków na sekundę dla rozwiązań wolnych od blokady.



Rysunek 3: Przepustowość w zależności od liczby filozofów

Tabela 1 zestawia mediany dla $n = 5$ i $n = 100$.

Scenariusz	Rozwiązanie	$n = 5$			$n = 100$		
		posiłki/s	czekanie [µs]	J	posiłki/s	czekanie [µs]	J
0/0	b) oba naraz	6 163 329	0,6	1,00	4 630 063	16,1	0,98
	c) lokaj	3 569 841	1,1	1,00	3 522 370	15,0	0,99
	asymetryczne	6 740 508	0,6	0,48	7 409 489	13,4	0,02
0/1	b) oba naraz	1668	1803,4	0,77	42 099	1213,0	0,50
	c) lokaj	853	4611,2	0,32	1555	67 170,1	0,04
	asymetryczne	906	4264,8	0,34	4147	24 532,0	0,14
1/1	b) oba naraz	1499	941,5	1,00	30 188	985,4	1,00
	c) lokaj	1258	1578,8	1,00	20 619	2528,9	1,00
	asymetryczne	1409	1185,9	0,99	21 616	2311,9	0,99
10/1	b) oba naraz	400	194,0	1,00	7695	145,9	1,00
	c) lokaj	395	221,7	1,00	7688	168,3	1,00
	asymetryczne	396	194,4	1,00	7688	160,8	1,00

Tabela 1: Przepustowość, średni czas oczekiwania na widelce i indeks Jaina J

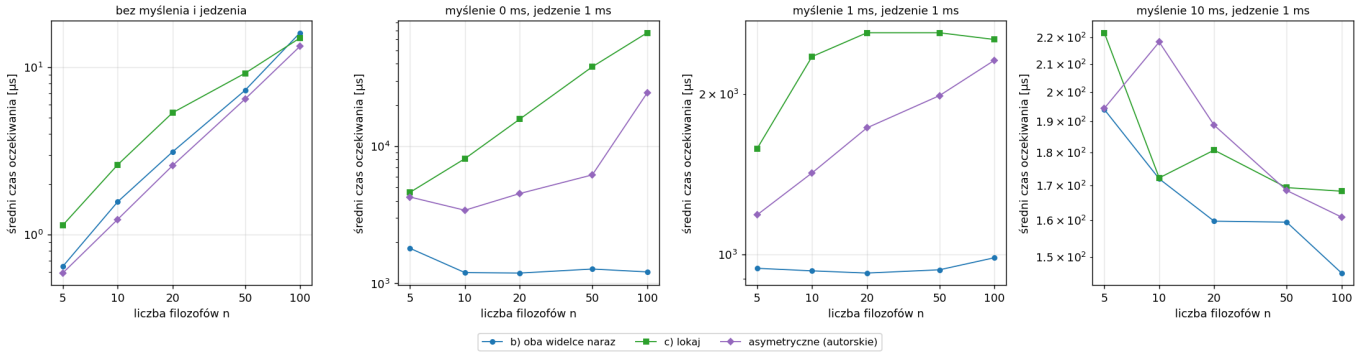
Bez myślenia i jedzenia (0/0) najszybsze jest rozwiązanie asymetryczne: 6,7–7,4 mln posiłków na sekundę, bo każdy widelec synchronizuje tylko dwóch sąsiadów i nie ma wspólnego monitora. Rozwiązanie b) jest wolniejsze (4,6–6,2 mln), a dla $n \geq 10$ wyraźnie traci, bo każdy posiłek przechodzi przez jeden monitor stołu, a `notifyAll()` budzi wszystkich czekających. Lokaj jest najwolniejszy (3,5 mln) – każdy posiłek to dwa wejścia do wspólnego monitora lokaja i cztery do monitorów widelców.

Przy jedzeniu 1 ms koszt synchronizacji jest pomijalny, a przepustowość zależy od tego, ilu filozofów je jednocześnie. W scenariuszu 10/1 filozofowie rzadko konkurują, wszystkie rozwiązania osiągają ten sam wynik, rosnący liniowo z n (nieco poniżej $\frac{n}{11\,\mathrm{ms}}$ ze względu na narzut `parkNanos`). W scenariuszu 1/1 najlepsze jest rozwiązanie b), bo filozof nie zajmuje widelca, z którego nie może skorzystać. W pozostałych rozwiązaniach filozof trzymający jeden widelec blokuje sąsiada, mimo że sam jeszcze nie je, i powstają łańcuchy oczekujących filozofów.

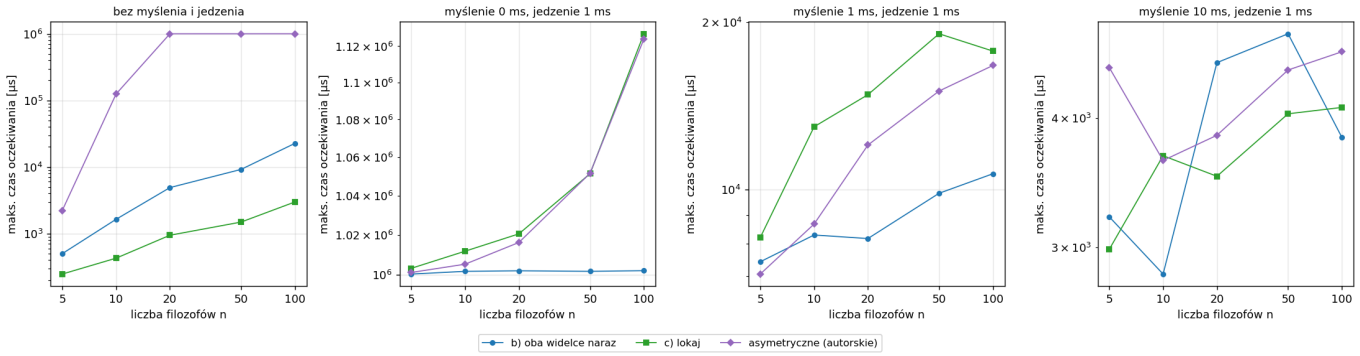
Największe różnice są w scenariuszu 0/1 (ciągle głodni filozofowie). Rozwiązanie b) osiąga dla $n = 100$ 42 tys. posiłków na sekundę, czyli blisko maksimum $\frac{n}{2}$ jedzących naraz (50 tys.). Rozwiązanie asymetryczne osiąga 4,1 tys., a lokaj tylko 1,6 tys. posiłków na sekundę – mniej niż 2 filozofów je jednocześnie. Filozof, który odłożył widelce, natychmiast próbuje podnieść je ponownie, więc niemal każdy widelec jest zajęty przez kogoś, kto czeka na drugi, i tworzą się długie łańcuchy oczekiwania. Przy lokaju dodatkowo kolejność wpuszczania do stołu nie ma związku z dostępnymi widelcami.

5.3 Czas oczekiwania

Rysunki 4 i 5 pokazują średni i maksymalny czas oczekiwania na widelce.



Rysunek 4: Średni czas oczekiwania na widelce

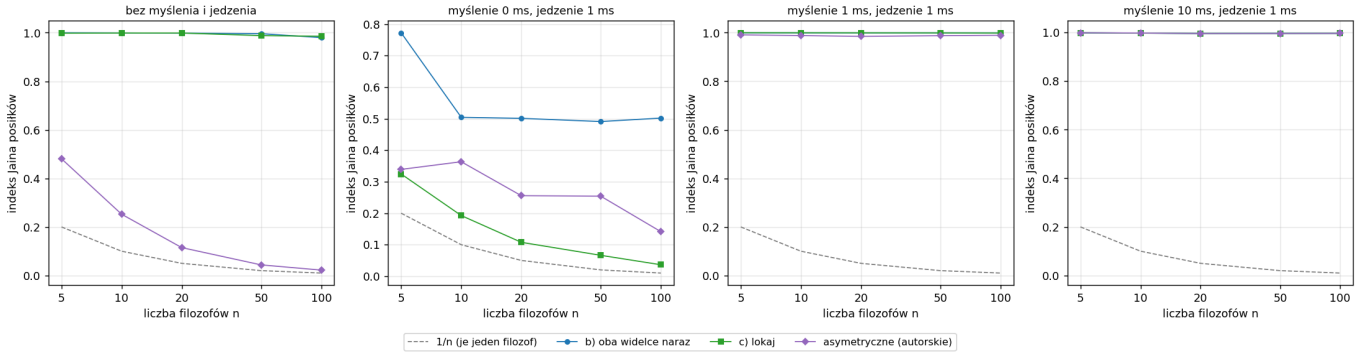


Rysunek 5: Maksymalny czas oczekiwania na widelce

Średnie czasy odpowiadają przepustowości: przy małej rywalizacji (10/1) filozof czeka średnio 150–220 µs, a w scenariuszu 0/1 dla $n = 100$ od 1,2 ms (b) do 67 ms (lokaj). Maksymalny czas oczekiwania w scenariuszu 0/1 dla wszystkich rozwiązań wynosi około 1 s, czyli cały przebieg: niektórzy filozofowie doczekali się widelców dopiero wtedy, gdy pozostali kończyli pracę. Bez myślenia i jedzenia to samo dotyczy rozwiązania asymetrycznego dla $n = 100$.

5.4 Sprawiedliwość

Rysunek 6 pokazuje indeks Jaina liczby posiłków. Przerywana linia $\frac{1}{n}$ odpowiada sytuacji, w której je tylko jeden filozof.



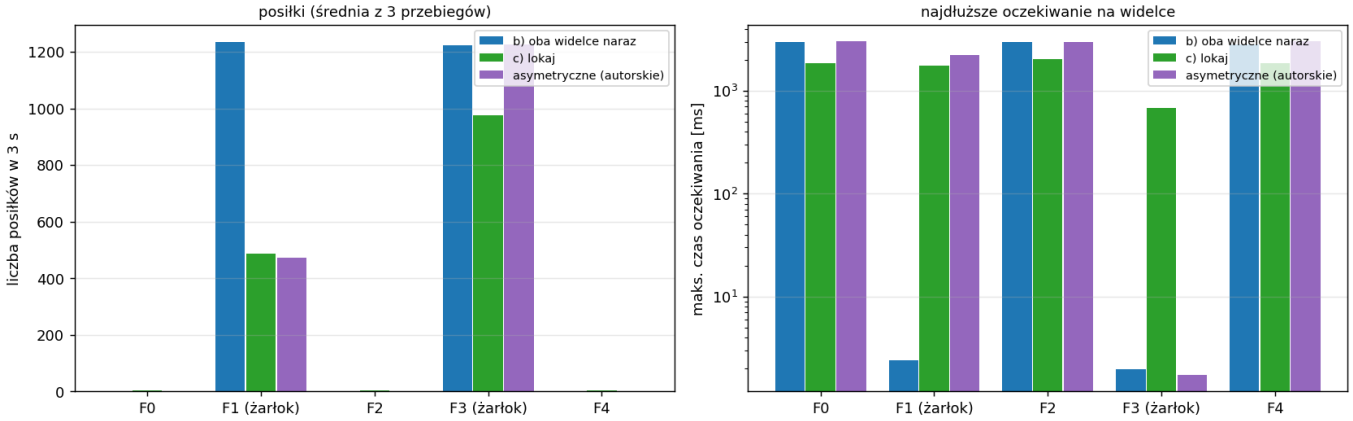
Rysunek 6: Indeks Jaina liczby posiłków

Gdy filozofowie myślą (1/1 i 10/1), wszystkie rozwiązania są sprawiedliwe ($J \geq 0{,}99$). Gdy nie myślą (0/1), wszystkie głodzą filozofów. W rozwiązaniu b) dla $n \geq 10$ $J \approx 0{,}5$: je co drugi filozof, a jego sąsiedzi prawie wcale. W rozwiązaniu z lokajem i asymetrycznym J spada dla $n = 100$ do 0,04 i 0,14, czyli blisko $\frac{1}{n}$. Bez myślenia i jedzenia (0/0) b) i lokaj są sprawiedliwe, a rozwiązanie asymetryczne nie (dla $n = 100$ $J = 0{,}02$).

Przyczyna jest ta sama co w laboratorium 4: monitor Javy nie gwarantuje kolejności wejścia. Filozof, który odłożył widelec (NOTIFY), od razu wywołuje TAKE ponownie i zwykle zdobywa monitor widelca, zanim obudzony sąsiad zdąży się wykonać. Przerwa na myślenie daje sąsiadowi czas na zabranie widelca, dlatego w scenariuszach 1/1 i 10/1 problem znika.

5.5 Zagłodzenie przez sąsiadów

Rysunek 7 pokazuje wynik eksperymentu z dwoma żarłocznymi filozofami F1 i F3.



Rysunek 7: Liczba posiłków i najdłuższe oczekiwanie, F1 i F3 nie myślą, $n = 5$, 3 s

W rozwiązaniu b) żarłoki zjadły średnio 1233 i 1224 razy, a F0, F2 i F4 tylko 2, 1 i 2 razy w ciągu 3 s. Najdłuższe oczekiwanie F2 wyniosło 3,00 s, czyli praktycznie cały przebieg. To dokładnie scenariusz zagłodzenia z rozdziału 3.2: F1 i F3 nie są sąsiadami, więc mogą jeść jednocześnie i na zmianę, a F2 potrzebuje chwili, w której nie je żaden z nich. W rozwiązaniu z lokajem i asymetrycznym F0, F2 i F4 również jedzą tylko kilka razy, ale żarłoki też tracą (472–976 posiłków), bo czekają na widelce trzymane przez sąsiadów, którzy czekają na drugi widelec. Żadne z badanych rozwiązań nie chroni przed zagłodzeniem, gdy niektórzy filozofowie nie dają innym szansy.

6 Wnioski

Rozwiązanie trywialne prowadzi do blokady. Przy dużej rywalizacji następuje ona w ułamku milisekundy, a przy długim myśleniu może nie wystąpić przez długi czas, co czyni błąd trudnym do wykrycia w testach.

Rozwiązanie z widelcami podnoszonymi jednocześnie nie może się zablokować, ale może zagłodzić filozofa, którego sąsiedzi jedzą na zmianę. Gdy jedzenie trwa dłużej niż synchronizacja, jest najwydajniejsze ze wszystkich, bo nikt nie trzyma widelca bez jedzenia. Jego słabością jest jeden monitor stołu i `notifyAll()`, co widać, gdy jedzenie jest krótkie, a filozofów dużo.

Lokaj usuwa blokadę najmniejszą zmianą rozwiązania trywialnego, ale jest najwolniejszy: wspólny monitor jest wąskim gardłem, a przy dużej rywalizacji ograniczenie do $n-1$ filozofów nie zapobiega długim łańcuchom oczekiwania na widelce.

Rozwiązanie asymetryczne nie ma wspólnego punktu synchronizacji i ma najmniejszy koszt synchronizacji, ale przy dużej rywalizacji jest niesprawiedliwe.

Na wydajność wpływają więc przede wszystkim: stopień rywalizacji (stosunek czasu myślenia do jedzenia), to, czy filozof może trzymać widelec bez jedzenia, obecność wspólnego monitora (stół w b), lokaj w c)) oraz liczba niepotrzebnie budzonych wątków. Gwarancji braku zagłodzenia nie daje żadne z rozwiązań opartych na monitorach Javy, bo nie zachowują one kolejności oczekujących. Uzyskanie jej wymagałoby kolejki FIFO lub priorytetu dla najdłużej głodnego filozofa, kosztem przepustowości, podobnie jak kolejka biletowa w laboratorium 4.

Literatura

[1] E.W. Dijkstra: *Hierarchical ordering of sequential processes*. Acta Informatica 1(2), 1971.

[2] E.G. Coffman, M. Elphick, A. Shoshani: *System Deadlocks*. ACM Computing Surveys 3(2), 1971.

[3] `https://home.agh.edu.pl/~funika/tw/lab5/`

[4] `https://docs.oracle.com/javase/specs/jls/se27/html/jls-17.html`